

PatchOrchestrator Platform

Enterprise Architecture & Operations Guide

For Technical Stakeholders and Operations Teams

CONFIDENTIAL - INTERNAL USE ONLY

PatchOrchestrator Enterprise Documentation

1. Main Overview

PatchOrchestrator -- Enterprise Patch Management Platform

v2.0.0 | *Spring Boot 3.2 Java 21 PostgreSQL WinRM/SSH CyberArk*

An enterprise-grade automated OS patch orchestration platform supporting Windows and Linux servers with role-based access control, CyberArk credential management, maintenance window enforcement, compliance tracking, and rich reporting.

Features

Category	Capabilities
Job Lifecycle	Create Approve Schedule Execute Monitor Retry
Remote Execution	WinRM (Windows) SSH (Linux) Mock (Demo/Dev)
Credential Security	CyberArk PAM integration -- credentials fetched live, never stored
Compliance	Per-server and per-environment compliance tracking with history
Maintenance Windows	Scheduled / weekly / one-time windows that gate job execution
Reporting	CSV export (jobs, audit, execution) PDF compliance report
Dashboard	Real-time KPIs, OS distribution chart, job status chart
RBAC	Admin (full control) User (create & view jobs)
Auditability	Immutable audit log for all system actions
Resilience	Configurable retry logic, timeout enforcement, Spring Retry

Quick Start (Demo Mode)

Prerequisites

- * Java 21+
- * Maven 3.9+
- * PostgreSQL 14+

1. Create the Database

PatchOrchestrator Enterprise Documentation

```
CREATE DATABASE patch_management;
```

2. Configure `application.yml`

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/patch_management
    username: YOUR_DB_USER
    password: YOUR_DB_PASSWORD
```

3. Run

```
mvn spring-boot:run -DskipTests
```

Visit: **http://localhost:8080**

4. Login

Role	Username	Password
Admin	`admin`	`Admin@123`
User	`user1`	`User@123`

Click the credential chip on the login page to auto-fill.

Demo mode is ON by default -- no external infrastructure (WinRM, SSH, CyberArk) is needed.

Architecture



PatchOrchestrator Enterprise Documentation

Credential Provider	Execution Engine	JPA / PostgreSQL
WinRmStrategy	10+ Repositories	
CyberArk (or Mock)	SshStrategy MockStrategy	Server PatchJob ComplianceRecord

Configuration Reference

All configuration lives in `src/main/resources/application.yml`.

Demo Mode (default: `true`)

```
integration:
  demo-mode: true    # Set to false for production
```

When `demo-mode: true`:

- * A **MockCredentialProvider** returns a simulated `root/password` credential
- * A **MockExecutionStrategy** simulates command execution (sleep + fake output)
- * No WinRM, SSH, or CyberArk connections are made

Switching to Production Mode

Step 1: Disable Demo Mode

```
integration:
  demo-mode: false
```

Step 2: Configure CyberArk

```
integration:
  cyberark:
    enabled: true
    base-url: https://your-cyberark-host/AIMWebService/api/Accounts
    app-id: PatchMgmtApp
    safe: PatchMgmtSafe
    connection-timeout-seconds: 10
```

PatchOrchestrator Enterprise Documentation

```
read-timeout-seconds: 15
verify-ssl: true      # Set false only for self-signed certs in lab
```

In CyberArk: Create an Application with ID `PatchMgmtApp` and grant it access to the safe containing server credentials.

In the app: Go to **Admin Credentials**, create a mapping for each server:

- * Select the server
- * Enter the CyberArk Safe name (e.g., `PatchMgmtSafe`)
- * Enter the CyberArk Object name (e.g., `WEB-PROD-01-admin`)
- * Optionally enter the CyberArk Username to filter by

During job execution, the platform calls the CyberArk AIM REST API to fetch credentials dynamically. Passwords are held in `char[]` and cleared immediately after use.

Step 3: Enable WinRM (for Windows Servers)

```
integration:
  winrm:
    enabled: true
    port: 5985          # 5986 for HTTPS
    use-https: false    # Set true for HTTPS
    timeout-seconds: 120
    auth-scheme: basic  # Options: basic | ntlm | kerberos
```

On each Windows Server, run:

```
# Enable WinRM
Enable-PSRemoting -Force
Set-Item WSMan:\localhost\Service\Auth\Basic -Value $true
Set-Item WSMan:\localhost\Service\AllowUnencrypted -Value $true # only for HTTP
netsh advfirewall firewall add rule name="WinRM-HTTP" dir=in localport=5985 protocol=TCP action=allow
```

On each server entity, set Execution Strategy to `WINRM`.

Step 4: Enable SSH (for Linux Servers)

```
integration:
  ssh:
    enabled: true
    port: 22
    timeout-seconds: 120
    strict-host-key-checking: false # Set true in prod with known-hosts file
```

On each Linux Server:

- * Ensure the patching user has `sudo` privileges (or runs as root)

PatchOrchestrator Enterprise Documentation

- * SSH service must be running and accessible from the application host
- * Port 22 (or custom) must be open in the firewall

On each server entity, set Execution Strategy to `SSH`.

Step 5: Set Server Execution Strategies

In the Server form (**Servers Edit**), set:

- * `WINRM` for Windows servers
- * `SSH` for Linux servers

Or configure globally in the database seeds / DataInitializer.

Job Workflow

```
[Create Job]  PENDING

[Admin Approves]  APPROVED

[Scheduler or Manual Execute]

[Maintenance Window Check]  BLOCKED (if outside window)

[Credential Fetch]  CyberArk / Mock

[Remote Command Execute]  WinRM / SSH / Mock  IN_PROGRESS

COMPLETED          FAILED

          [Retry if retryCount < maxRetries]  RETRYING

          [max retries exceeded]  FAILED permanently
          (always)

[Compliance Record Saved]  Server status updated
```

Patch Commands executed per OS:

OS	Command Template
Windows	`wusa.exe <patch.msu> /quiet /norestart` or custom `installCommand`
Linux	`sudo apt-get install -y <package>` or `sudo yum install -y <package>` or custom

PatchOrchestrator Enterprise Documentation

Reporting

Report	Format	Endpoint
Patch Job Report	CSV	`GET /reports/export/patch-csv?environment=PROD&status=COMPLETED`
Compliance Report	PDF	`GET /reports/export/compliance-pdf?environment=PROD`
Audit Log Export	CSV	`GET /reports/export/audit-csv`
Execution Log Export	CSV	`GET /reports/export/execution-csv`

REST API

The platform exposes a REST API for integration with external tools:

Endpoint	Method	Description
`/api/v1/jobs`	GET	List all patch jobs
`/api/v1/jobs/{id}`	GET	Get job details
`/api/v1/jobs/{id}/status`	GET	Get job status (used for polling)
`/api/v1/jobs/{id}/execute`	POST	Trigger job execution
`/api/v1/servers`	GET	List servers
`/api/v1/compliance`	GET	Get compliance summary
`/actuator/health`	GET	Application health

Security

Area	Implementation
Authentication	Spring Security form login with BCrypt password hashing
Authorization	`ROLE_ADMIN` (full) / `ROLE_USER` (create + view)
CSRF	Enabled on all state-changing requests
Credentials	Stored as `char[]` and cleared after use -- never logged
CyberArk	TLS-verified connection; App-ID based authentication
Session	8-hour timeout; invalidated on logout

PatchOrchestrator Enterprise Documentation

Area	Implementation
Actuator	Restricted to `ROLE_ADMIN` only

Patch Engine Settings

```
patch:
  scheduler:
    enabled: true
    cron: "0 * * * * *"          # Every minute -- check for due jobs
  execution:
    max-retry-attempts: 3
    retry-delay-seconds: 30
    batch-size: 5
    default-timeout-minutes: 30
    thread-pool-size: 10
```

Jobs execute asynchronously. Per-job retry and timeout can be overridden in the job's configuration.

Project Structure

```
src/main/java/com/patchmgmt/
  config/           # AsyncConfig, Security, DataInitializer, Scheduler
  controller/       # MVC + REST controllers
  dto/              # Data transfer objects
  entity/           # JPA entities (10)
  enums/            # PatchStatus, OsType, ComplianceStatus,
  exception/        # Custom exceptions
  integration/
    credential/     # CyberArk + Mock credential providers
    execution/      # WinRM + SSH + Mock execution strategies
    model/          # ExecutionResult, ResolvedCredential
  repository/       # Spring Data JPA repositories (10)
  service/          # Service interfaces + implementations (11)

src/main/resources/
  application.yml   # All configuration
  static/
    css/style.css   # Enterprise dark glassmorphism theme
    js/app.js        # Chart.js, polling, live clock
  templates/        # Thymeleaf templates (25+)
```

PatchOrchestrator Enterprise Documentation

Data Seeded in Demo Mode

On first startup, the following sample data is created automatically:

- * **2 users:** `admin` (ADMIN role), `user1` (USER role)
- * **5 servers:** Mix of Windows and Linux across PROD/DEV/STAGING
- * **6 patches:** Critical/High/Medium severity for both Windows and Linux
- * **3 patch jobs:** In various states (PENDING, COMPLETED, FAILED)
- * **3 maintenance windows:** Weekly PROD window, DEV daily window, One-time patch
- * **4 environments:** PROD, DEV, STAGING, UAT
- * **Compliance records:** Pre-seeded for all servers

Troubleshooting

Issue	Fix
`500 on startup`	Check PostgreSQL is running and credentials in `application.yml` are correct
`CyberArk connection refused`	Verify `cyberark.base-url` and network access; check App-ID permissions
`WinRM authentication failure`	Ensure Basic auth is enabled on Windows target: `Set-Item WSMan:\localhost\Service\Auth\Basic -Value \$true`
`SSH Permission denied`	Verify the CyberArk credential resolves to a user with SSH access
`Job stuck IN_PROGRESS`	Check async thread pool; increase `thread-pool-size` if under heavy load
`Compliance not updating`	Compliance is recorded after each job execution -- run at least one job per server

License

Proprietary -- Internal enterprise use only.

Enterprise Patch Orchestration Platform Built with Spring Boot 3.2 Java 21

2. Orchestration Guide

Enterprise Patch Orchestration Platform -- Complete Guide

What Is It?

A **Spring Boot web application** that lets your team securely deploy OS patches to remote Linux/Windows servers.

It enforces a **4-eyes approval policy** (someone creates a job, someone else approves it), tracks compliance, streams live execution logs, and supports both package-manager commands and manual file upload for patching.

Architecture Overview

flowchart TD

```
Browser[" Browser (Thymeleaf UI)"]
SC["Spring Security\n(BCrypt  CSRF  Role-based)"]
CTRL["Controllers Layer\n(Patch / Job / Server / Admin / API)"]
SVC["Service Layer\n(PatchService  PatchJobService  ComplianceService)"]
SCHED["PatchJobScheduler\n(@Scheduled -- every minute)"]
EXEC["RemoteExecutionStrategy\n(interface)"]
SSH["SshExecutionStrategy\n(Apache Sshd)"]
WINRM["WinRmExecutionStrategy\n(PowerShell Invoke-Command)"]
MOCK["MockExecutionStrategy\n(Demo / Simulation)"]
DISP["DispatchingExecutionStrategy\n(routes by OS type at runtime)"]
CRED["CredentialProvider\n(CyberArk REST  or  Local AES-256-GCM)"]
DB["PostgreSQL\n(JPA / Hibernate)"]
UPLOAD["patch-uploads/\n(local binary store)"]
```

```
Browser --> SC --> CTRL --> SVC
SCHED --> SVC
SVC --> CRED
SVC --> EXEC
EXEC --> DISP
DISP --> |LINUX| SSH
DISP --> |WINDOWS| WINRM
DISP --> |demo-mode=true| MOCK
SVC --> DB
CTRL --> DB
CTRL --> UPLOAD
```

PatchOrchestrator Enterprise Documentation

Key Layers Explained

1. Presentation (Thymeleaf + Vanilla CSS)

Template Folder	Purpose
`auth/`	Login page
`dashboard/`	Home -- summary cards, recent jobs
`servers/`	Register / manage target servers
`patches/`	Patch catalog -- create, edit, file-upload
`jobs/`	Create jobs, live log viewer, retry
`admin/`	Admin Approvals Hub -- approve/reject jobs & deletion requests
`compliance/`	Compliance dashboard per server/patch
`maintenance/`	Maintenance window schedule configuration
`reports/`	Reporting views

2. Security (`SecurityConfig`)

- * **BCrypt** password hashing
- * **CSRF** protection on all POST/multipart forms
- * **Role-based access:**

Role	Access
`ROLE_ADMIN`	Full access -- `/admin/`, `/maintenance/`, `/reports/`, approve/delete
`ROLE_USER`	Create jobs & patches, view logs, view compliance

Default seeded accounts:

Username	Password	Role
`admin`	`Admin@123`	ADMIN
`user1`	`User@123`	USER
`devops`	`DevOps@123`	USER

3. Execution Engine (`integration/execution/`)

PatchOrchestrator Enterprise Documentation

This is the core engine that connects to remote servers.

```
RemoteExecutionStrategy (interface)
    execute(server, command, credential)          -- run a command
    executeWithFile(server, local, remote, cmd, credential) -- transfer file + run
    implemented by
    SshExecutionStrategy      Apache Sshd   Linux servers
    WinRmExecutionStrategy    PowerShell Invoke-Command  Windows servers
    MockExecutionStrategy     simulated output -- safe for demos
    selected by
    DispatchingExecutionStrategy (picks SSH / WinRM / Mock at runtime based on server.osType)
```

File upload execution flow (Linux `.sh`):

```
1. mkdir -p /tmp/patches/          (remote dir)
2. cat > /tmp/patches/nano_install.sh (stream file via SSH stdin)
3. sed -i 's/\r//' /tmp/patches/nano_install.sh (strip Windows CRLF)
4. echo 'pw' | sudo -S bash /tmp/patches/nano_install.sh (run as root)
```

4. Credential Management (``integration/credential``)

Mode	Config	How it works
Demo mode	<code>`integration.demo-mode: true`</code>	Mock -- no real credentials needed
Local	<code>`cyberark.enabled: false`</code>	Per-server credentials stored AES-256-GCM encrypted in <code>`credential_mappings`</code> table
CyberArk	<code>`cyberark.enabled: true`</code>	Fetches secrets from CyberArk AIM REST API at execution time

5. Scheduling (``PatchJobScheduler``)

Runs **every minute** (cron: ``0 * * *``).

Finds all ``APPROVED`` jobs whose ``scheduledAt <= now`` and executes them asynchronously via the ``patchExecutor`` thread pool (size configurable, default 10).

6. Database Schema (PostgreSQL)

Table	Description
<code>`app_users`</code>	Users with roles
<code>`servers`</code>	Target servers (IP, OS, ports, environment)

PatchOrchestrator Enterprise Documentation

Table	Description
`patches`	Patch catalog (commands, file paths, severity)
`patch_jobs`	Job instances (status, scheduled time, logs)
`execution_logs`	Per-attempt detailed execution records
`compliance_records`	COMPLIANT / NON_COMPLIANT per server+patch
`credential_mappings`	Encrypted SSH/WinRM credentials per server
`maintenance_windows`	Scheduled patching windows
`audit_logs`	Full audit trail of all actions
`deletion_requests`	4-eyes deletion workflow

How to Use It -- Step by Step

Step 1 -- Configure `application.yml`

```
# Real SSH mode (no demo, no CyberArk):
integration:
  demo-mode: false
  cyberark:
    enabled: false
  ssh:
    enabled: true
    port: 22
    timeout-seconds: 120
  local-credentials:
    encryption-key: "YourSecretPassphrase-ChangeMe"

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/patch_management
    username: postgres
    password: yourpassword
```

Step 2 -- Start the Application

```
mvn spring-boot:run
# or
java -jar target/patch-management-2.0.0.jar
```

Open: **http://localhost:8080** Login as `admin / Admin@123`

PatchOrchestrator Enterprise Documentation

Step 3 -- Register a Server

Servers Add New Server

Field	Example
Name	`Kali Test 2`
IP Address	`192.168.56.102`
OS Type	`LINUX`
SSH Port	`22`
Environment	`DEV`

Then go to **Credential Mappings** and add:

- * Username: `kali`
- * Password: (encrypted and stored)

Step 4 -- Create a Patch

Patches Add New Patch

Option A -- Catalogue / Package Manager Patch

Field	Example
Title	`Install curl 8.4.0`
Patch ID	`CVE-2023-38545`
OS Type	`LINUX`
Severity	`MEDIUM`
Install Command	`apt-get update && apt-get install -y curl`

Leave the file upload section empty.

Option B -- Manual File Upload Patch

Field	Example
Title	`Install axel via script`
OS Type	`LINUX`
Severity	`LOW`
Patch binary	Upload `axel_install.sh`
Install command	*(auto-suggested: `chmod +x {FILE} && bash {FILE}`)*

PatchOrchestrator Enterprise Documentation

The system auto-suggests the install command when you pick a file.

`{FILE}` is replaced at execution time with the full remote path.

Supported file types:

OS	Extensions	Auto-generated Command
Linux	`.sh`	`chmod +x {FILE} && bash {FILE}`
Linux	`.deb`	`DEBIAN_FRONTEND=noninteractive dpkg -i {FILE}`
Linux	`.rpm`	`rpm -Uvh {FILE}`
Windows	`.msu`	`wusa.exe "{FILE}" /quiet /norestart`
Windows	`.exe`	`"{FILE}" /silent /norestart`
Windows	`.msi`	`msiexec.exe /i "{FILE}" /quiet /norestart`

Step 5 -- Create a Patch Job

Jobs New Job

Field	Example
Title	`Deploy axel to Kali Test 2`
Server	`Kali Test 2`
Patch	`Install axel via script`
Scheduled At	*(Set a time, or blank = immediate on approval)*

Job is created in **PENDING** state.

Step 6 -- Approve the Job (4-Eyes Policy)

Admin Approvals Hub

An admin (or a different user) reviews and clicks **Approve**.

The job moves to **APPROVED** state.

The scheduler picks it up within 1 minute if `scheduledAt` has passed.

Or the executor can manually trigger it immediately via "Execute Now."

Step 7 -- Monitor Execution (Live Logs)

PatchOrchestrator Enterprise Documentation

Jobs View Job Execution Log

The log streams in real time. Example output for a file-upload job:

```
[2026-04-13T12:11:27] PATCH JOB EXECUTION STARTED
[2026-04-13T12:11:27] Patch type      : MANUAL FILE UPLOAD
[2026-04-13T12:11:27] Local file     : ./patch-uploads/1776062_nano_install.sh
[2026-04-13T12:11:27] Remote deploy  : /tmp/patches/nano_install.sh
[2026-04-13T12:11:27] Install command : chmod +x {FILE} && bash {FILE}
[2026-04-13T12:11:27] Starting file transfer + install via SSH...
[Upload] File transferred to /tmp/patches/nano_install.sh
[Install] echo '*****' | sudo -S bash /tmp/patches/nano_install.sh
Installing axel...
axel 2.17.11
Installation complete.
Patch execution SUCCESSFUL (exit code: 0)
```

Step 8 -- Compliance & Reporting

- * **Compliance** dashboard shows COMPLIANT / NON_COMPLIANT per server
- * Successful jobs automatically mark the server as `COMPLIANT`
- * Failed jobs mark `NON\_COMPLIANT`
- * **Reports** section provides audit history and compliance summaries

Configuration Reference

Key `application.yml` Knobs

Property	Default	Description
`integration.demo-mode`	`false`	`true` = safe simulation, no real connections
`integration.ssh.enabled`	`true`	Enable real SSH execution
`integration.ssh.timeout-seconds`	`120`	SSH command timeout
`integration.cyberark.enabled`	`false`	Enable CyberArk secret fetch
`integration.local-credentials.encryption-key`	*(changeme)*	AES-256 key for stored credentials
`patch.upload.dir`	`./patch-uploads`	Where uploaded binaries are stored
`patch.scheduler.enabled`	`true`	Enable cron-based auto-execution
`spring.servlet.multipart.max-file-size`	`1024MB`	Max upload size for patch binaries

PatchOrchestrator Enterprise Documentation

API Endpoints (REST)

Method	Path	Description
`GET`	`/api/v1/jobs/{id}/status`	Poll job status (PENDING/IN_PROGRESS/COMPLETED/FAILED)
`GET`	`/api/v1/jobs/{id}/logs`	Fetch execution log for live viewer
`GET`	`/api/v1/patches/suggest-command?osType=LINUX&filename=patch.sh`	Get auto-suggested install command
`GET`	`/api/v1/ping`	Health check
`GET`	`/actuator/health`	Spring Boot health endpoint

Common Issues & Fixes

Symptom	Cause	Fix	
`File-based execution not supported`	`DispatchingExecutionStrategy` missing `executeWithFile`	Fixed	
`chmod: cannot access 'file.sh': No such file or directory`	Install command uses wrong filename	Use `{FILE}` placeholder or leave blank to auto-detect	
`\$'\r': command not found`	Script has Windows CRLF line endings	Auto-stripped via `sed` post-upload	
`sudo: a terminal is required`	SSH exec channel has no TTY -- sudo can't prompt	Script is run as `echo 'pw' \	sudo -S bash {FILE}`
`duplicate key value -- patch_id already exists`	Leaving Patch ID blank sent `""na""` every time	Auto-generates `UPLOAD-<timestamp>` when blank	
Job status shows SUCCESS but software not installed	sudo failed silently, script exited 0	Fixed by sudo -S wrapper; check STDERR in logs	

3. Component Workflow

Application Component Workflow & Architecture

This document explains the internal working of different components in the PatchOrchestrator application.

1. Dashboard: How Data is Fetched

The Dashboard provides a real-time overview of the system state.

- * **Frontend Layer:** Built using Thymeleaf templates (`index.html`) with embedded JavaScript (Chart.js) for visualization.
- * **Controller Layer:** The `DashboardController` handles the `/dashboard` endpoint.
- * **Service & Repository Layer:** The controller calls multiple services:
 - * `ServerService.getServerStats()`: Queries the `ServerRepository` to count total servers, and groups them by OS type and Environment.
 - * `PatchJobService.getJobStats()`: Queries the `PatchJobRepository` to get counts of jobs in different states (PENDING, APPROVED, IN_PROGRESS, COMPLETED, FAILED).
 - * `ComplianceService.getOverallCompliance()`: Queries `ComplianceRecordRepository` to get the latest compliance status of the fleet.
- * **Data Flow:** The aggregated data is added to the Spring MVC `Model` and rendered server-side by Thymeleaf. The rendered HTML is sent to the browser. Any live updates are fetched via AJAX calls to REST endpoints (`/api/v1/jobs/status`).

2. Patch Jobs: Creation, Storage, and Execution

Patch jobs are the core entities that track a patching operation.

- * **Creation:**

1. A user (or Admin) navigates to "Create Job" in the UI.
2. They submit a form containing the Server ID, Patch ID, and Scheduled Time.
3. The `PatchJobController` receives the HTTP POST request, validates it, and converts it to a `PatchJobDTO`.
4. It calls `PatchJobService.createJob(dto)`.

- * **Storage:**

- * The service instantiates a `PatchJob` JPA Entity.
- * It saves the entity using `PatchJobRepository.save()`.
- * The data is stored in the `patch\_jobs` table in the PostgreSQL database. The initial status is set to `PENDING`.

- * **Approval Workflow:**

- * `PENDING` jobs must be approved by a user with the `ROLE\_ADMIN` role. Once approved, the status changes to `APPROVED`.

PatchOrchestrator Enterprise Documentation

- * **Execution:**
- * A background Spring `@Scheduled` task (in `JobScheduler`) runs every minute to find `APPROVED` jobs whose scheduled time has arrived.
- * Alternatively, jobs can be manually triggered via the UI ("Execute Now").

3. Remote Execution: SSH and WinRM

When a job executes, the application needs to connect to the target server. This is handled by the `integration/execution` module.

- * **Execution Strategy Pattern:** The system uses the `RemoteExecutionStrategy` interface. Based on the target OS, a specific strategy is chosen:
- * **Windows (`WinRmExecutionStrategy`):** Uses Java's `ProcessBuilder` to launch a PowerShell process on the host running the app. It executes an `Invoke-Command` cmdlet using WinRM to connect to the target Windows server. It authenticates using credentials fetched from CyberArk.
- * **Linux (`SshExecutionStrategy`):** Uses the `Apache MINA SSHD` client library. It establishes a secure SSH connection to the target server on port 22, authenticates using CyberArk credentials, and executes the package manager command (e.g., `apt-get` or `yum`) in an exec channel.
- * **Security:** Passwords are never stored in the database. During execution, the `CredentialMappingService` calls the `CyberArkCredentialProvider` (which hits the CyberArk AIM REST API) to fetch the credential dynamically. The password is kept in a `char[]` and cleared immediately after use.

4. Demo Mode vs. Real Mode

The application can run in two distinct modes, controlled by a single configuration flag in `application.yml`:

```
integration:
  demo-mode: true # Switch to 'false' for Real Mode
```

- * **Demo Mode (`demo-mode: true`):**
- * Designed for local development, testing, and demonstrations without needing external infrastructure.
- * **Credential Provider:** `MockCredentialProvider` is injected. It returns fake hardcoded credentials instead of calling CyberArk.
- * **Execution Strategy:** `MockExecutionStrategy` is injected. Instead of making real SSH/WinRM network connections, it uses `Thread.sleep()` to simulate execution delay and returns fake success logs.
- * **Data Initialization:** The `DataInitializer` bean automatically seeds the database with sample users, servers, patches, and historical jobs.
- * **Real Mode (`demo-mode: false`):**
- * Designed for Production.
- * **Credential Provider:** `CyberArkCredentialProvider` is injected. It makes live REST calls to your organization's CyberArk instance.
- * **Execution Strategy:** `WinRmExecutionStrategy` and `SshExecutionStrategy` are injected. They make

PatchOrchestrator Enterprise Documentation

real network connections to target servers and execute commands.

5. Maintenance Windows

To ensure patches are only applied during approved downtimes:

- * When a job is triggered, the `MaintenanceWindowService` is consulted.
- * It checks if the current time falls within any active maintenance window defined for the target server's environment.
- * If no active window is found, the job execution is blocked.

6. Audit & Compliance

- * **Audit Log:** Every significant action (Job Created, Job Approved, Job Executed, Status Changed) is recorded via the `AuditLogService` into the `audit\_logs` table. This provides an immutable history.
- * **Compliance:** After a patch job finishes (successfully or failed), the `ComplianceService` updates the `compliance\_records` table. This tracking drives the Compliance Dashboard and PDF reports.

4. Execution Architecture

Remote Execution & Credential Architecture

Here is the architectural flow illustrating how the Patch Management system interacts with credentials, determines the operating system, and executes safely configured processes across Windows and Linux.

```
graph TD
    A["Patch Job Triggered"] --> B["Fetch Target Server details from DB"]

    subgraph Credential Retrieval
        C["CyberArkCredentialProvider"]
        D["CyberArk AIM REST API"]
    end

    B -->|"Requires Credential"| C
    C -->|"HTTP GET Request (AppID, Safe, Object)"| D
    D -->|"Returns UserName & Password payload"| C

    C -->|"ResolvedCredential"| E["ExecutionStrategyFactory Check OsType"]

    %% Windows Flow
    subgraph Windows Branch ["Windows Execution via WinRM"]
        E -- OsType.WINDOWS --> WES["WinRmExecutionStrategy"]
        WES --> W_Pre["Stop IIS/AppPool (ServiceLifecycleManager)"]
        W_Pre --> W_Chunk["Convert patch to Base64 (32KB Upload Chunks)"]
        W_Chunk --> W_Invoke["Execute native cmd.exe to powershell.exe Invoke-Command"]
        W_Invoke --> W_Exec["Stream over WinRM Port 5985/5986 (Execute Install Command)"]
        W_Exec --> W_Post["Start IIS/AppPool (ServiceLifecycleManager)"]
    end

    %% Linux Flow
    subgraph Linux Branch ["Linux Execution via SSH"]
        E -- OsType.LINUX --> SES["SshExecutionStrategy"]
        SES --> L_Pre["Stop systemctl service (ServiceLifecycleManager)"]
        L_Pre --> L_Stream["Connect via Apache MINA SSHD (Stream patch via cat)"]
        L_Stream --> L_Dos["Run sed command to strip Windows/DOS line endings"]
        L_Dos --> L_Exec["Execute script via 'echo pw | sudo -S sh'"]
        L_Exec --> L_Post["Start systemctl service (ServiceLifecycleManager)"]
    end

    W_Post --> X["Update execution status to COMPLETED/FAILED"]
    L_Post --> X

    %% Styling
    classDef default fill:#f9f9f9,stroke:#333,stroke-width:2px;
```

PatchOrchestrator Enterprise Documentation

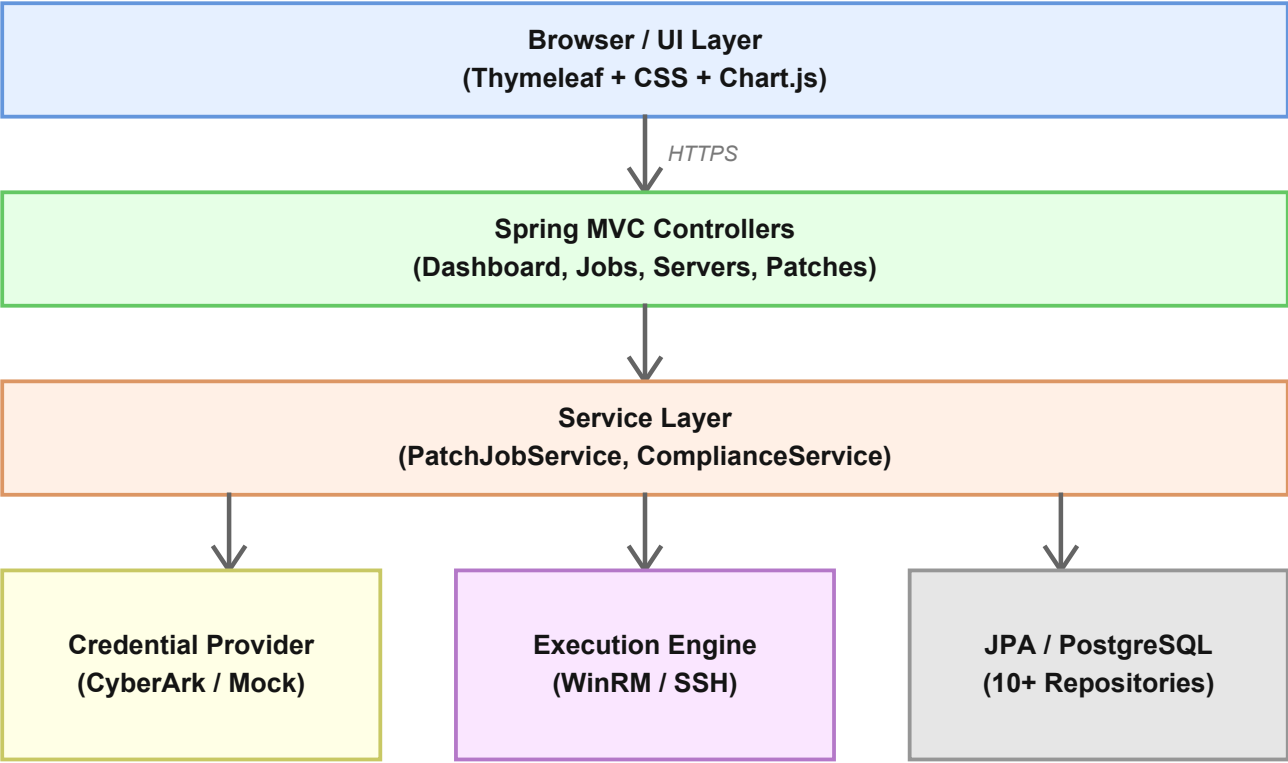
```
classDef success fill:#d4edda,stroke:#28a745,stroke-width:2px;
classDef highlight fill:#cfe2f3,stroke:#17a2b8,stroke-width:2px;

class C,D highlight;
class X success;
```

Key Components

1. **Database to CyberArk:** A target server is picked up. Its properties refer to a specific logical 'Safe' and 'Object' mapped in the DB.
2. **Execution Strategy Routing:** The `OsType` enumerator strictly enforces how the `ResolvedCredential` will be used.
3. **Windows WinRM flow:** Base64 encodes files to circumvent sluggish SMB networking limits, forcing traffic natively over HTTP/HTTPS WinRM.
4. **Linux SSH flow:** Circumvents SCP dependencies completely by cleanly wrapping file drops with the standard `cat` command, automatically transforming carriage returns to Linux-friendly ones, and handling `sudo` without physical terminal windows.

5. Logical Architecture Diagram



6. Remote Execution Workflow Diagram

